AG TECHNOLOGIES

Architecture de Services Partagés

Cahier des Charges Technique

Service d'Abonnement Centralisé

Subscription Service — AGT-Subscription

Version	1.2
Date	Avril 2026
Statut	Implementation-ready
Classification	Confidentiel
Auteur	Équipe Architecture
Dépendances	Auth v2.1, Users v1.0, Notification v1.2

Document confidentiel — Réservé à l'usage interne d'AG Technologies

Historique des Révisions

Ver.	Date	Auteur	Description
1.0	2026-04-03	Équipe Archi.	Création initiale. Modèle hybride B2C+B2B via subscriber abstrait. Plans, quotas, cycles, trial, prorata, overage, organisations. Contrats inter-services formalisés. Diagramme PlantUML en annexe.
1.1	2026-04-03	Équipe Archi.	Post-audit : quota atomique (reserve/confirm/release). Idempotence événements RabbitMQ (`event_id`, `idempotency_key`). Stratégie fallback/mode dégradé (4 niveaux). Retry crons avec backoff exponentiel + dead letter queue. Immutabilité des plans actifs (versioning). Snapshot quota au billing overage. Table `quota_reservations`.
1.2	2026-04-03	Équipe Archi.	Version finale : statut *Implementation-ready*. Contrats inter-services alignés sur reserve/confirm/release pour chemins critiques. Résidus documentaires corrigés.

Table des matières

1 Introduction **4**
1.1 Périmètre . . . 4
1.2 Hors périmètre . . . 4

2 Contexte & Problématique **4**
2.1 Contexte . . . 4
2.2 Problématique . . . 4

3 Vision & Objectifs **4**
3.1 Vision . . . 5
3.2 Concept du subscriber abstrait . . . 5
3.3 Objectifs . . . 5

4 Parties Prenantes & Utilisateurs **5**

5 Besoins Fonctionnels **6**
5.1 Gestion des Plans . . . 6
5.2 Gestion des Quotas . . . 7
5.3 Souscription & Cycle de Vie . . . 8
5.4 Trial . . . 8
5.5 Upgrade & Downgrade . . . 9
5.6 Organisations (B2B) . . . 9
5.7 Configuration Plateforme . . . 9
5.8 Administration . . . 10

6 Besoins Non-Fonctionnels **11**

7 Architecture Technique **12**
7.1 Vue d'ensemble . . . 12
7.2 Stack Technique . . . 12
7.3 Dépendances & Règles d'Architecture . . . 12
7.4 Stratégie de résilience (v1.1) . . . 12
7.5 Diagramme d'architecture . . . 13

8 Modélisation **13**
8.1 Diagramme de Classes . . . 13
8.2 MLD — Modèle Logique de Données . . . 13
8.2.1 Table `plans` . . . 14
8.2.2 Table `plan_prices` . . . 14
8.2.3 Table `plan_quotas` . . . 14
8.2.4 Table `organizations` . . . 15
8.2.5 Table `organization_members` . . . 15
8.2.6 Table `subscriptions` . . . 15
8.2.7 Table `subscription_quotas_usage` . . . 16
8.2.8 Table `quota_reservations` (v1.1) . . . 16
8.2.9 Table `subscription_events` . . . 17
8.2.10 Table `platform_subscription_config` . . . 17

9 Contrats d'API **17**
9.1 Health Check . . . 17
9.1.1 GET /subscriptions/health . . . 17

9.2 Plans . . . 18
9.2.1 POST /subscriptions/plans . . . 18
9.2.2 GET /subscriptions/plans . . . 18
9.2.3 GET /subscriptions/plans/{planId} . . . 18
9.2.4 PUT /subscriptions/plans/{planId} . . . 19
9.2.5 POST /subscriptions/plans/{planId}/archive . . . 19
9.3 Souscriptions . . . 19
9.3.1 POST /subscriptions . . . 19
9.3.2 GET /subscriptions/{subId} . . . 19
9.3.3 GET /subscriptions . . . 19
9.3.4 POST /subscriptions/{subId}/cancel . . . 19
9.3.5 POST /subscriptions/{subId}/reactivate . . . 20
9.4 Changement de Plan . . . 20
9.4.1 POST /subscriptions/{subId}/change-plan . . . 20
9.5 Quotas . . . 20
9.5.1 POST /subscriptions/quotas/check . . . 20
9.5.2 POST /subscriptions/quotas/increment . . . 21
9.5.3 POST /subscriptions/quotas/reserve (v1.1) . . . 22
9.5.4 POST /subscriptions/quotas/confirm (v1.1) . . . 22
9.5.5 POST /subscriptions/quotas/release (v1.1) . . . 23
9.5.6 GET /subscriptions/{subId}/usage . . . 23
9.6 Organisations . . . 23
9.6.1 POST /subscriptions/organizations . . . 23
9.6.2 POST /subscriptions/organizations/{orgId}/members . . . 23
9.6.3 DELETE /subscriptions/organizations/{orgId}/members/{userId} . . . 23
9.6.4 GET /subscriptions/organizations/{orgId}/members . . . 23
9.7 Configuration Plateforme . . . 23
9.7.1 PUT /subscriptions/config/{platformId} . . . 23
9.7.2 GET /subscriptions/config/{platformId} . . . 24
9.8 Administration . . . 24
9.8.1 GET /subscriptions/admin/stats . . . 24
9.8.2 POST /subscriptions/admin/{subId}/suspend . . . 24
9.8.3 POST /subscriptions/admin/{subId}/force-change . . . 24
9.8.4 DELETE /subscriptions/by-user/{userId} . . . 24

10 Contrats Inter-Services **24**
10.1 Abonnement → Auth (JWT utilisateur) . . . 25
10.2 Abonnement → Auth (token S2S) . . . 25
10.3 Abonnement → Users (résolution subscriber) . . . 25
10.4 Abonnement → Notification (alertes) . . . 25
10.5 Services consommateurs → Abonnement (vérification quotas) (v1.2) . . . 26
10.6 Abonnement → Paiement (événements RabbitMQ) (v1.1) . . . 26
10.7 Users → Abonnement (Purge RGPD) . . . 27

11 Matrice d'Autorisation **27**

12 Recommandations & Évolutions **27**

1 Introduction

Ce document constitue le cahier des charges du **Service d'Abonnement Centralisé** (ci-après « Subscription Service ») destiné à l'écosystème AGT. Il sert de référence unique pour la conception, le développement et les décisions techniques.

1.1 Périmètre

Le service Abonnement couvre la gestion complète du cycle de vie des abonnements : définition de plans avec quotas, souscription, renouvellement, upgrade/downgrade avec prorata, trial, dépassement (overage), grâce après expiration, et vérification de quotas en temps réel pour les services consommateurs.

1.2 Hors périmètre

- Authentification (déléguée au Service Auth v2.1)
- Profils utilisateur, rôles et permissions (délégués au Service Users v1.0)
- Exécution des paiements (déléguée au futur Service Paiement)
- Stockage de fichiers (délégué au Service Média v1.4)
- Envoi de notifications (délégué au Service Notification v1.2)
- Logique métier propre aux plateformes consommatrices

Convention d'identité

Dans tout ce document :
- `user_id` désigne `users_auth.id` (= `sub` du JWT)
- `platform_id` désigne un UUID du registre Auth (`platforms.id`)
- Abonnement ne stocke aucune table locale de plateformes ni d'utilisateurs. Il consomme Auth et Users.

2 Contexte & Problématique

2.1 Contexte

Plateforme	Statut	Besoin abonnement
AGT-Bot	En développement	Plans par nombre de bots, messages IA/mois
AGT-Market	En développement	Plans vendeur par nombre de produits, boosts
MboaMove	En développement	Plans chauffeur, dispatch prioritaire
SALMA	Migration prévue	Plans étudiant, accès formations
Futures	Planifié	Toute plateforme nécessitant une monétisation

2.2 Problématique

Sans service centralisé, chaque plateforme implémenterait sa propre gestion d'abonnements, entraînant : duplication de la logique de quotas, incohérence dans la gestion des cycles de vie, impossibilité de mutualiser les règles de facturation, et absence de vision unifiée sur les revenus récurrents.

3 Vision & Objectifs

3.1 Vision

Un service **générique et abstrait** qui gère les abonnements pour toute plateforme de l'écosystème. Il ne connaît aucun concept métier : il gère des plans, des quotas, des cycles et des souscriptions.

Principe fondamental

Le service Abonnement **définit les règles** (qui a droit à quoi, en quelle quantité, pendant combien de temps). Il ne facture rien : c'est le futur Service Paiement qui exécute les transactions. Abonnement **décide**, Paiement **exécute**.

3.2 Concept du subscriber abstrait

L'abonné n'est pas forcément un individu. Le service modélise un **subscriber** abstrait :

subscriber_type	subscriber_id pointe vers	Exemple
`user`	`users_auth.id`	Yannick s'abonne seul au plan Pro AGT-Bot
`organization`	`organizations.id` (table locale)	TechCorp s'abonne au plan Enterprise, 15 membres partagent les quotas

Chaque plateforme peut utiliser uniquement le mode `user` au début, et activer le mode `organization` quand elle en a besoin. Aucune obligation.

3.3 Objectifs

#	Objectif	Indicateur
O1	Gestion multi-plateforme	Tout plan est scopé à une plateforme
O2	Modèle hybride	Forfait + dépassement configurable par quota
O3	Subscriber abstrait	Support B2C (user) et B2B (organization)
O4	Cycle de vie complet	Trial, actif, grâce, suspendu, annulé, expiré
O5	Prorata	Upgrade/downgrade calculé au jour
O6	Vérification temps réel	Endpoint quota check $<$ 50ms
O7	Flexibilité maximale	Trial, grâce, post-trial, plan gratuit : tout configurable par plateforme

4 Parties Prenantes & Utilisateurs

Rôle	Type	Responsabilité
Product Owner	Interne	Priorisation des fonctionnalités
Équipe Backend	Interne	Développement et maintenance
Équipe Frontend	Interne	Intégration UI plans, upgrade, quotas
DevOps	Interne	Infrastructure, monitoring, crons
Admin global AGT	Interne	Gestion transverse des plans, limites système
Admin plateforme	Interne	Gestion des plans de sa plateforme
Utilisateurs finaux	Externe	Souscription, upgrade, consultation quotas
Services internes	Interne	Chatbot, Média, Search : vérification quotas

5 Besoins Fonctionnels

5.1 Gestion des Plans

ID	Fonctionnalité	Description	Priorité
F-PLN-01	Création de plan	Créer un plan avec nom, prix, cycle, quotas, pour une plateforme	**Urgent**
F-PLN-02	Liste des plans	Lister les plans d'une plateforme (publics et/ou internes)	**Urgent**
F-PLN-03	Détail d'un plan	Consulter un plan avec ses quotas et features	**Urgent**
F-PLN-04	Modification de plan	Modifier prix, quotas, features (n'affecte pas les abonnements actifs)	**Urgent**
F-PLN-05	Archivage de plan	Désactiver un plan (plus de nouvelles souscriptions, existants maintenus)	**Urgent**
F-PLN-06	Plan gratuit par défaut	Configurable par plateforme : un plan peut être marqué `is_default`	Normale
F-PLN-07	Hiérarchie admin	Admin global peut tout faire ; admin plateforme limité à sa plateforme	**Urgent**
F-PLN-08	Immutabilité plans actifs	Un plan avec abonnements actifs est immutable sur prix/quotas. Créer un nouveau plan et archiver l'ancien.	**Urgent**

Versioning des plans (v1.1)

Règle : un plan ayant au moins un abonnement actif (statut $\neq$ expired, cancelled) est **immutable** sur ses champs `plan_prices` et `plan_quotas`. Seuls `name`, `description` et `metadata` restent modifiables.
Pour changer les prix ou quotas : créer un nouveau plan (avec `tier_order` identique ou supérieur), archiver l'ancien. Les abonnés existants restent sur leur plan jusqu'au renouvellement ou upgrade volontaire. Aucune migration automatique.

5.2 Gestion des Quotas

ID	Fonctionnalité	Description	Priorité
F-QTA-01	Définition quotas par plan	Chaque plan définit N quotas avec clé, limite, et politique (hard/overage)	**Urgent**
F-QTA-02	Vérification quota	Endpoint S2S : un service demande « cet abonné a-t-il encore du quota X ? »	**Urgent**
F-QTA-03	Incrémentation usage	Endpoint S2S : un service reporte une consommation sur un quota	**Urgent**
F-QTA-04	Consultation usage	L'abonné consulte sa consommation courante vs limites	**Urgent**
F-QTA-05	Reset périodique	Les compteurs de quotas cycliques sont remis à zéro au renouvellement	**Urgent**
F-QTA-06	Politique hard limit	Quota atteint = accès bloqué. Le service appelant reçoit un refus	**Urgent**
F-QTA-07	Politique overage	Quota atteint = dépassement autorisé, comptabilisé pour facturation	Normale
F-QTA-08	Quotas libres par plateforme	Les clés de quotas sont définies librement (pas de liste codée en dur)	**Urgent**
F-QTA-09	Snapshot fin de cycle	À chaque fin de cycle, snapshot de l'usage et overage persisté avant reset. Base de la facturation overage.	**Urgent**

Exemples de clés de quotas

- AGT-Bot : `max_bots`, `messages_per_month`, `storage_mb`
- AGT-Market : `max_products`, `boosts_per_month`
- MboaMove : `priority_dispatch` (booléen via quota 0/1)
- SALMA : `max_active_courses`, `certifications_included`

Les clés sont des `VARCHAR` libres, scopées par plateforme. Abonnement ne connaît pas leur signification métier.

5.3 Souscription & Cycle de Vie

ID	Fonctionnalité	Description	Priorité
F-SUB-01	Souscription	Créer un abonnement (subscriber + plan + cycle)	**Urgent**
F-SUB-02	Activation post-paiement	Passer de `pending_payment` à `active` sur confirmation Paiement	**Urgent**
F-SUB-03	Renouvellement auto	Cron : tenter le renouvellement avant expiration via Paiement	**Urgent**
F-SUB-04	Expiration	Cron : passer en `grace` puis `expired` selon config plateforme	**Urgent**
F-SUB-05	Annulation	L'abonné annule ; reste actif jusqu'à fin du cycle payé	**Urgent**
F-SUB-06	Suspension	Admin ou système suspend un abonnement (impayé, abus)	Normale
F-SUB-07	Réactivation	Reprendre un abonnement suspendu ou expiré	Normale
F-SUB-08	Historique	Consulter l'historique complet des changements d'état	Normale

Cycle de vie d'un abonnement

```
pending_payment --> active --> [renouvellement OK] --> active
                      |
                      +--> cancelled (par l'abonne, actif jusqu'a fin cycle)
                      |
                      +--> [expiration] --> grace --> expired --> suspended
                      |
                      +--> suspended (par admin/systeme)

trial --> active (si paiement)
      --> [selon config plateforme] : downgrade_to_free | suspended | expired
```

5.4 Trial

ID	Fonctionnalité	Description	Priorité
F-TRI-01	Activation trial	Souscription avec période d'essai (durée configurable par plateforme)	**Urgent**
F-TRI-02	Fin de trial	Comportement post-trial configurable : downgrade, suspend, ou expire	**Urgent**
F-TRI-03	Conversion trial → payant	Si paiement reçu pendant le trial, passage en `active`	**Urgent**
F-TRI-04	Un seul trial par plan	Un subscriber ne peut pas bénéficier deux fois du trial sur le même plan	Normale

5.5 Upgrade & Downgrade

ID	Fonctionnalité	Description	Priorité
F-CHG-01	Upgrade	Passage à un plan supérieur en cours de cycle avec prorata au jour	**Urgent**
F-CHG-02	Downgrade	Passage à un plan inférieur en cours de cycle avec prorata au jour	**Urgent**
F-CHG-03	Calcul prorata	Crédit restant sur l'ancien plan − coût restant sur le nouveau plan	**Urgent**
F-CHG-04	Historique changements	Chaque changement de plan est loggé avec montants prorata	Normale

Calcul du prorata

Upgrade : jours restants sur l'ancien plan × prix/jour ancien = crédit. Jours restants × prix/jour nouveau = débit. Différence = montant à facturer via Paiement.
Downgrade : même logique inversée. Le crédit résiduel est stocké et déduit du prochain cycle.

5.6 Organisations (B2B)

ID	Fonctionnalité	Description	Priorité
F-ORG-01	Création organisation	Créer une organisation avec nom, owner, plateforme	**Urgent**
F-ORG-02	Ajout de membres	Ajouter un user_id comme membre d'une organisation	**Urgent**
F-ORG-03	Retrait de membres	Retirer un membre (l'owner ne peut pas être retiré)	**Urgent**
F-ORG-04	Rôles organisation	Owner (admin) et member (consommateur de quotas)	Normale
F-ORG-05	Quotas partagés	Les quotas de l'abonnement org sont mutualisés entre membres	**Urgent**
F-ORG-06	Liste membres	Consulter les membres d'une organisation	Normale

5.7 Configuration Plateforme

ID	Fonctionnalité	Description	Priorité
F-CFG-01	Config trial	Durée trial par défaut, comportement post-trial	**Urgent**
F-CFG-02	Config grâce	Durée du délai de grâce après expiration	**Urgent**
F-CFG-03	Config plan par défaut	Quel plan est assigné par défaut (ou aucun)	Normale
F-CFG-04	Config devise	Devise des prix pour cette plateforme	**Urgent**
F-CFG-05	Config cycles autorisés	Quels cycles de facturation sont proposés	Normale

5.8 Administration

ID	Fonctionnalité	Description	Priorité
F-ADM-01	Stats par plateforme	Nombre d'abonnés actifs, MRR, churn, par plan	Normale
F-ADM-02	Liste abonnements	Filtrage par plateforme, plan, statut, subscriber	**Urgent**
F-ADM-03	Forcer un changement	Admin peut forcer upgrade, downgrade, suspension, annulation	**Urgent**
F-ADM-04	Limites système	Admin global définit des plafonds	Normale

6 Besoins Non-Fonctionnels

ID	Catégorie	Exigence	Cible
NF-01	Performance	Vérification quota (endpoint check)	< 50ms (P95)
NF-02	Performance	Temps de réponse API	< 200ms (P95)
NF-03	Performance	Throughput vérification quotas	1000 req/s min
NF-04	Disponibilité	Uptime	99.9%
NF-05	Sécurité	Auth obligatoire	JWT ou S2S
NF-06	Sécurité	Rate limiting	Configurable
NF-07	Scalabilité	Horizontal	Stateless, multi-instance
NF-08	Observabilité	Logs structurés	JSON, request ID
NF-09	Observabilité	Health check	`GET /subscriptions/health`
NF-10	Données	Cache quotas	Redis TTL 30s
NF-11	Données	Timestamps	TIMESTAMPTZ
NF-12	Traitement	Cron renouvellement/expiration	Celery Beat, retry 3x backoff (1min, 5min, 15min), dead letter queue

7 Architecture Technique

7.1 Vue d'ensemble

Le service Abonnement est un microservice REST stateless, déployé en conteneur Docker. Il expose une API JSON consommée par les plateformes clientes et les services internes (vérification de quotas).

7.2 Stack Technique

Composant	Technologie
Langage & Framework	Python 3.11+ / Django 5.x
API	Django REST Framework (DRF)
Base de données	PostgreSQL 15+
ORM	Django ORM
Migrations	Django Migrations
Cache	Redis 7+ (quotas, vérifications fréquentes)
Scheduler	Celery + Celery Beat
Message Broker	RabbitMQ
Validation	DRF Serializers
Conteneurisation	Docker + Docker Compose
CI/CD	GitHub Actions

7.3 Dépendances & Règles d'Architecture

Service	Type	Usage
Service Auth	Interne	Validation JWT via `GET /auth/verify-token`. Validation S2S via `POST /auth/s2s/introspect`. Registre plateformes.
Service Users	Interne	Résolution `user_id`, vérification statut actif. Cache Redis TTL 60s.
Service Notification	Interne	Alertes : expiration, renouvellement, fin trial, quota atteint.
Service Paiement	Interne (futur)	Exécution paiements. Confirmation via événement RabbitMQ.

Règle stricte de dépendance

Abonnement ne dépend d'aucun service métier. Il est consommé par les plateformes et les services internes (Chatbot, Média, Search) pour la vérification de quotas. Le couplage est unidirectionnel : les services interrogent Abonnement, jamais l'inverse.
Exception : Abonnement émet des événements RabbitMQ consommés par Paiement (demande de facturation) et Notification (alertes cycle de vie).

7.4 Stratégie de résilience (v1.1)

Le service Abonnement est sur le **chemin critique** de tout l'écosystème (chaque message IA, chaque upload, chaque boost passe par la vérification de quotas). Une indisponibilité d'Abonnement bloquerait Chatbot, Média et Search simultanément.

Mode dégradé — Fallback Strategy

Niveau 1 — Cache Redis (nominal) : les quotas et usages sont cachés dans Redis (TTL 30s). 95% des appels sont servis depuis le cache sans toucher PostgreSQL.
Niveau 2 — Cache local appelant (dégradé) : chaque service consommateur (Chatbot, Média) **doit** maintenir un cache local (in-memory, TTL 5 min) du dernier résultat de `quota/check`. Si Abonnement est injoignable, le service utilise ce cache local.
Niveau 3 — Grace allowance (failover) : si le cache local est également vide (premier appel, redémarrage), le service consommateur applique une **grace allowance** configurable : il autorise N requêtes (défaut : 10) avant de bloquer. Ces consommations sont reportées en batch dès qu'Abonnement redevient disponible.
Niveau 4 — Soft enforcement (dernier recours) : en cas d'indisponibilité prolongée (> 5 min), les services passent en mode `soft enforcement` : les quotas sont loggés mais non bloqués. Une alerte critique est envoyée à l'équipe ops via Notification.

Contrat de résilience pour les services consommateurs

Tout service appelant `quotas/check` ou `quotas/reserve` **doit** implémenter les niveaux 2 et 3. Ce n'est pas optionnel. Le non-respect de cette règle crée un SPOF sur l'écosystème.

7.5 Diagramme d'architecture

```
AGT-Bot   AGT-Market   MboaMove   SALMA   Futures
   |          |            |         |        |
   +-----+----+-------+----+----+----+--------+
         |            |         |
  Subscription Service (API REST)
    |         |          |           |
Auth(JWT)  Users(status)  Redis    PostgreSQL
    |
RabbitMQ -----> Paiement (futur)
    |-----> Notification (alertes)
```

8 Modélisation

8.1 Diagramme de Classes

Le diagramme de classes complet est fourni en annexe au format PlantUML (voir section 12). Il couvre les entités : `Plan`, `PlanPrice`, `PlanQuota`, `Organization`, `OrganizationMember`, `Subscription`, `SubscriptionQuotasUsage`, `QuotaReservation`, `SubscriptionEvent` et `PlatformSubscriptionConfig`.

8.2 MLD — Modèle Logique de Données

8.2.1 Table plans

Colonne	Type	Contrainte	Description
id	UUID	PK	Identifiant unique
platform_id	UUID	NOT NULL	Plateforme (UUID Auth)
name	VARCHAR(100)	NOT NULL	Nom du plan
slug	VARCHAR(100)	NOT NULL	Slug URL-friendly
description	TEXT	NULL	Description
is_active	BOOLEAN	DEFAULT true	Plan actif
is_default	BOOLEAN	DEFAULT false	Plan par défaut
is_free	BOOLEAN	DEFAULT false	Plan gratuit
tier_order	INT	DEFAULT 0	Ordre hiérarchique
metadata	JSONB	NULL	Données complémentaires
created_at	TIMESTAMPTZ	NOT NULL	Création
updated_at	TIMESTAMPTZ	NOT NULL	Dernière mise à jour

Contrainte unique : `(platform_id, slug)`.

8.2.2 Table plan_prices

Colonne	Type	Contrainte	Description
id	UUID	PK	Identifiant unique
plan_id	UUID	FK → plans	Plan associé
billing_cycle	VARCHAR(20)	NOT NULL	monthly, yearly, custom
cycle_days	INT	NULL	Jours si custom
price	DECIMAL(12,2)	NOT NULL	Prix du cycle
currency	VARCHAR(3)	NOT NULL	Devise (XAF, EUR, USD)
is_active	BOOLEAN	DEFAULT true	Prix actif
created_at	TIMESTAMPTZ	NOT NULL	Création

Contrainte unique : `(plan_id, billing_cycle, currency)`.

8.2.3 Table plan_quotas

Colonne	Type	Contrainte	Description
id	UUID	PK	Identifiant unique
plan_id	UUID	FK → plans	Plan associé
quota_key	VARCHAR(50)	NOT NULL	Clé libre (`max_bots`, `messages_per_month`)
limit_value	INT	NOT NULL	Limite (-1 = illimité)
is_cyclical	BOOLEAN	DEFAULT true	RAZ à chaque cycle
overage_policy	VARCHAR(10)	DEFAULT 'hard'	`hard` ou `overage`
overage_unit_price	DECIMAL(10,2)	DEFAULT 0	Prix unitaire du dépassement

Contrainte unique : `(plan_id, quota_key)`.

8.2.4 Table `organizations`

Colonne	Type	Contrainte	Description
id	UUID	PK	Identifiant unique
platform_id	UUID	NOT NULL	Plateforme
name	VARCHAR(150)	NOT NULL	Nom de l'organisation
owner_user_id	UUID	NOT NULL	Propriétaire (`users_auth.id`)
is_active	BOOLEAN	DEFAULT true	Active
created_at	TIMESTAMPTZ	NOT NULL	Création
updated_at	TIMESTAMPTZ	NOT NULL	Dernière MAJ

Contrainte unique : `(platform_id, name)`.

8.2.5 Table `organization_members`

Colonne	Type	Contrainte	Description
id	UUID	PK	Identifiant unique
organization_id	UUID	FK → organizations	Organisation
user_id	UUID	NOT NULL	Membre (`users_auth.id`)
role	VARCHAR(20)	DEFAULT 'member'	`owner` ou `member`
joined_at	TIMESTAMPTZ	NOT NULL	Date d'ajout

Contrainte unique : `(organization_id, user_id)`.

8.2.6 Table `subscriptions`

Colonne	Type	Contrainte	Description
id	UUID	PK	Identifiant unique
platform_id	UUID	NOT NULL	Plateforme
subscriber_type	VARCHAR(20)	NOT NULL	`user` ou `organization`
subscriber_id	UUID	NOT NULL	user_id ou organization_id
plan_id	UUID	FK → plans	Plan souscrit
plan_price_id	UUID	FK → plan_prices	Prix/cycle choisi
status	VARCHAR(20)	NOT NULL	pending_payment, trial, active, grace, expired, suspended, cancelled
current_period_start	TIMESTAMPTZ	NOT NULL	Début du cycle courant
current_period_end	TIMESTAMPTZ	NOT NULL	Fin du cycle courant
trial_end	TIMESTAMPTZ	NULL	Fin de la période d'essai
grace_end	TIMESTAMPTZ	NULL	Fin du délai de grâce
cancelled_at	TIMESTAMPTZ	NULL	Date d'annulation
cancel_at_period_end	BOOLEAN	DEFAULT false	Annulation en fin de cycle
created_at	TIMESTAMPTZ	NOT NULL	Création
updated_at	TIMESTAMPTZ	NOT NULL	Dernière MAJ

Contrainte unique partielle : `(platform_id, subscriber_type, subscriber_id) WHERE status NOT IN ('expired','cancelled')`.

Règle d'unicité

Un subscriber ne peut avoir qu'**un seul abonnement actif** par plateforme. Les abonnements terminés (expired, cancelled) restent en base pour l'historique.

8.2.7 Table `subscription_quotas_usage`

Colonne	Type	Contrainte	Description
id	UUID	PK	Identifiant unique
subscription_id	UUID	FK → subscriptions	Abonnement
quota_key	VARCHAR(50)	NOT NULL	Clé du quota
used	INT	DEFAULT 0	Consommation courante
overage	INT	DEFAULT 0	Dépassement
period_start	TIMESTAMPTZ	NOT NULL	Début période
period_end	TIMESTAMPTZ	NOT NULL	Fin période
updated_at	TIMESTAMPTZ	NOT NULL	Dernière MAJ

Contrainte unique : `(subscription_id, quota_key, period_start)`.

Snapshot de fin de cycle (v1.1)

Avant le reset des quotas cycliques, le cron de renouvellement crée un `subscription_event` de type `cycle_closed` avec dans `metadata` un snapshot complet : `{quota_key, limit, used, overage}` pour chaque quota. Ce snapshot est la source de vérité pour la facturation overage. Le reset n'intervient qu'**après** la création du snapshot.

8.2.8 Table `quota_reservations` (v1.1)

Colonne	Type	Contrainte	Description
id	UUID	PK	Identifiant unique (= reservation_id)
subscription_id	UUID	FK → subscriptions	Abonnement
quota_key	VARCHAR(50)	NOT NULL	Clé du quota
amount	INT	NOT NULL	Quantité réservée
status	VARCHAR(20)	DEFAULT 'pending'	pending, confirmed, released, expired
expires_at	TIMESTAMPTZ	NOT NULL	Expiration de la réservation
created_at	TIMESTAMPTZ	NOT NULL	Création
resolved_at	TIMESTAMPTZ	NULL	Date de confirm/release/expiration

Index : `(subscription_id, quota_key, status)` pour lookup rapide des réservations pending.

8.2.9 Table subscription_events

Colonne	Type	Contrainte	Description
id	UUID	PK	Identifiant unique
subscription_id	UUID	FK → subscriptions	Abonnement
event_type	VARCHAR(30)	NOT NULL	created, activated, renewed, upgraded, downgraded, cancelled, expired, suspended, reactivated, cycle_closed
from_plan_id	UUID	NULL	Plan précédent
to_plan_id	UUID	NULL	Nouveau plan
prorate_credit	DECIMAL(12,2)	NULL	Crédit prorata
prorate_debit	DECIMAL(12,2)	NULL	Débit prorata
metadata	JSONB	NULL	Données complémentaires
created_at	TIMESTAMPTZ	NOT NULL	Date de l'événement

8.2.10 Table platform_subscription_config

Colonne	Type	Contrainte	Description
id	UUID	PK	Identifiant unique
platform_id	UUID	UNIQUE, NOT NULL	Plateforme
default_trial_days	INT	DEFAULT 0	Durée trial (0 = pas de trial)
grace_period_days	INT	DEFAULT 0	Délai de grâce (0 = coupure immédiate)
post_trial_behavior	VARCHAR(20)	DEFAULT 'suspend'	`downgrade_to_free`, `suspend`, `expire`
default_currency	VARCHAR(3)	DEFAULT 'XAF'	Devise par défaut
allowed_cycles	JSONB	NOT NULL	Cycles autorisés
require_default_plan	BOOLEAN	DEFAULT false	Exiger un plan par défaut
created_at	TIMESTAMPTZ	NOT NULL	Création
updated_at	TIMESTAMPTZ	NOT NULL	Dernière MAJ

9 Contrats d'API

Base URL : `/api/v1`. Tous les endpoints retournent du JSON. Les endpoints protégés requièrent `Authorization: Bearer <token>`.

9.1 Health Check

9.1.1 GET /subscriptions/health

Vérifie l'état du service. Aucune authentification requise.

Response 200 :

```
{
  "status": "healthy",
  "database": "ok",
  "redis": "ok",
  "version": "1.2.0"
```

```
}
```

9.2 Plans

9.2.1 POST /subscriptions/plans

Création d'un plan. Auth : Bearer token (admin plateforme ou admin global).

Request body :

```
{
  "platform_id": "uuid",
  "name": "Pro",
  "slug": "pro",
  "description": "Plan professionnel",
  "is_free": false,
  "tier_order": 2,
  "prices": [
    { "billing_cycle": "monthly", "price": 15000, "currency": "XAF" },
    { "billing_cycle": "yearly", "price": 150000, "currency": "XAF" }
  ],
  "quotas": [
    {
      "quota_key": "max_bots",
      "limit_value": 5,
      "is_cyclical": false,
      "overage_policy": "hard"
    },
    {
      "quota_key": "messages_per_month",
      "limit_value": 10000,
      "is_cyclical": true,
      "overage_policy": "overage",
      "overage_unit_price": 2
    }
  ]
}
```

Response 201 :

```
{
  "id": "plan-uuid",
  "platform_id": "uuid",
  "name": "Pro",
  "slug": "pro",
  "message": "Plan created"
}
```

Codes : 201 Created, 400 Validation error, 401 Unauthorized, 403 Forbidden, 409 Slug exists.

9.2.2 GET /subscriptions/plans

Liste des plans. Query params : `?platform_id=uuid&is_active=true&page=1&limit=20`.

Codes : 200 OK, 401 Unauthorized.

9.2.3 GET /subscriptions/plans/{planId}

Détail d'un plan avec prix et quotas. Codes : 200 OK, 404 Not found.

9.2.4 PUT /subscriptions/plans/{planId}

Modification. N'affecte pas les abonnements actifs. Codes : 200 OK, 400, 403, 404.

9.2.5 POST /subscriptions/plans/{planId}/archive

Archivage (`is_active = false`). Codes : 200 OK, 403, 404.

9.3 Souscriptions

9.3.1 POST /subscriptions

Création d'un abonnement.

Request body :

```
{
  "platform_id": "uuid",
  "subscriber_type": "user",
  "subscriber_id": "user-uuid",
  "plan_id": "plan-uuid",
  "billing_cycle": "monthly",
  "with_trial": true
}
```

Response 201 :

```
{
  "id": "sub-uuid",
  "status": "trial",
  "plan": { "id": "plan-uuid", "name": "Pro" },
  "current_period_start": "2026-04-03T00:00:00Z",
  "current_period_end": "2026-05-03T00:00:00Z",
  "trial_end": "2026-04-17T00:00:00Z",
  "message": "Subscription created"
}
```

Codes : 201 Created, 400 Validation, 404 Plan not found, 409 Active sub exists.

9.3.2 GET /subscriptions/{subId}

Détail avec plan, quotas, usage courant. Codes : 200, 404.

9.3.3 GET /subscriptions

Liste. Query params : `?platform_id&subscriber_id&status&page&limit`. Codes : 200, 401.

9.3.4 POST /subscriptions/{subId}/cancel

Annulation. Actif jusqu'à fin du cycle.

Response 200 :

```
{
  "id": "sub-uuid",
  "status": "active",
  "cancel_at_period_end": true,
  "message": "Subscription will be cancelled at period end"
}
```

Codes : 200, 404, 409 Already cancelled.

9.3.5 POST /subscriptions/{subId}/reactivate

Réactivation. Codes : 200, 404, 409 Cannot reactivate.

9.4 Changement de Plan

9.4.1 POST /subscriptions/{subId}/change-plan

Upgrade ou downgrade avec prorata.

Request body :

```
{
  "new_plan_id": "plan-uuid",
  "billing_cycle": "monthly"
}
```

Response 200 :

```
{
  "id": "sub-uuid",
  "old_plan": { "id": "old-uuid", "name": "Free" },
  "new_plan": { "id": "new-uuid", "name": "Pro" },
  "prorate_credit": 0,
  "prorate_debit": 10000,
  "amount_due": 10000,
  "message": "Plan changed, payment required"
}
```

Codes : 200, 400 Same plan, 404 Plan not found, 409 No active sub.

9.5 Quotas

9.5.1 POST /subscriptions/quotas/check

Endpoint critique S2S. Vérifie si un subscriber a du quota disponible. Auth : S2S token.

Request body :

```
{
  "platform_id": "uuid",
  "subscriber_type": "user",
  "subscriber_id": "user-uuid",
  "quota_key": "messages_per_month",
  "requested": 1
}
```

Response 200 (disponible) :

```
{
  "allowed": true,
  "quota_key": "messages_per_month",
  "limit": 10000,
  "used": 4523,
  "remaining": 5477,
  "overage_policy": "overage"
}
```

Response 200 (épuisé, hard limit) :

```
{
  "allowed": false,
  "quota_key": "max_bots",
  "limit": 5,
  "used": 5,
  "remaining": 0,
  "overage_policy": "hard",
  "message": "Quota exceeded"
}
```

Codes : 200, 400, 401, 404 No active sub.

Performance quota check

Cet endpoint est sur le chemin critique de tous les services. Il **doit** répondre en < 50ms. Stratégie : cache Redis avec les quotas/usage du subscriber, TTL 30s, invalidé à chaque incrémentation.

9.5.2 POST /subscriptions/quotas/increment

Endpoint S2S. Reporte une consommation.

Request body :

```
{
  "platform_id": "uuid",
  "subscriber_type": "user",
  "subscriber_id": "user-uuid",
  "quota_key": "messages_per_month",
  "amount": 1
}
```

Response 200 :

```
{
  "quota_key": "messages_per_month",
  "used": 4524,
  "limit": 10000,
  "overage": 0
}
```

Codes : 200, 400, 403 Hard limit exceeded, 404 No active sub.

Atomicité des quotas (v1.1)

Le pattern `check` → `increment` en deux appels séparés est vulnérable aux race conditions (requêtes concurrentes). Le service implémente un modèle **reserve** / **confirm** / **release** :

1. **Reserve** : réserve atomiquement N unités (`UPDATE ... WHERE used + requested <= limit` ou autorise l'overage). Retourne un `reservation_id` avec TTL (défaut : 60s).
2. **Confirm** : le service appelant confirme la consommation. La réservation devient définitive.
3. **Release** : si le traitement échoue, libération de la réservation. Un cron purge les réservations expirées.

Implémentation : opération SQL atomique (`SELECT ... FOR UPDATE` ou `UPDATE ...`

`RETURNING`). Cache Redis invalidé après chaque reserve/confirm/release.
Rétrocompatibilité : `check` et `increment` restent disponibles pour les cas simples. Pour les chemins critiques à haute concurrence (messages IA, uploads), les services **doivent** utiliser reserve/confirm/release.

9.5.3 POST /subscriptions/quotas/reserve (v1.1)

Endpoint S2S. Réserve atomiquement du quota. Auth : S2S token.

Request body :

```
{
  "platform_id": "uuid",
  "subscriber_type": "user",
  "subscriber_id": "user-uuid",
  "quota_key": "messages_per_month",
  "amount": 1,
  "ttl_seconds": 60
}
```

Response 200 :

```
{
  "reservation_id": "res-uuid",
  "allowed": true,
  "quota_key": "messages_per_month",
  "limit": 10000,
  "used": 4524,
  "reserved": 1,
  "expires_at": "2026-04-03T12:01:00Z"
}
```

Codes : 200, 400, 403 Hard limit exceeded, 404 No active sub.

9.5.4 POST /subscriptions/quotas/confirm (v1.1)

Confirme une réservation. Auth : S2S token.

Request body :

```
{
  "reservation_id": "res-uuid"
}
```

Response 200 :

```
{
  "confirmed": true,
  "quota_key": "messages_per_month",
  "used": 4524
}
```

Codes : 200, 404 Reservation not found, 410 Reservation expired.

9.5.5 POST /subscriptions/quotas/release (v1.1)

Libère une réservation non utilisée. Auth : S2S token.

Request body :

```
{
  "reservation_id": "res-uuid"
}
```

Response 200 :

```
{
  "released": true,
  "quota_key": "messages_per_month",
  "used": 4523
}
```

Codes : 200, 404, 410 Already confirmed or expired.

9.5.6 GET /subscriptions/{subId}/usage

Usage courant de tous les quotas. Codes : 200, 404.

9.6 Organisations

9.6.1 POST /subscriptions/organizations

Request body :

```
{
  "platform_id": "uuid",
  "name": "TechCorp",
  "owner_user_id": "user-uuid"
}
```

Codes : 201, 400, 409 Name exists.

9.6.2 POST /subscriptions/organizations/{orgId}/members

Request body : `{"user_id": "uuid", "role": "member"}`

Codes : 201, 404, 409 Already member.

9.6.3 DELETE /subscriptions/organizations/{orgId}/members/{userId}

Codes : 200, 403 Cannot remove owner, 404.

9.6.4 GET /subscriptions/organizations/{orgId}/members

Codes : 200, 404.

9.7 Configuration Plateforme

9.7.1 PUT /subscriptions/config/{platformId}

Auth : admin global ou admin plateforme.

Request body :

```
{
  "default_trial_days": 14,
  "grace_period_days": 7,
  "post_trial_behavior": "downgrade_to_free",
  "default_currency": "XAF",
  "allowed_cycles": ["monthly", "yearly"],
  "require_default_plan": true
}
```

Codes : 200, 400, 403.

9.7.2 GET /subscriptions/config/{platformId}

Codes : 200, 404.

9.8 Administration

9.8.1 GET /subscriptions/admin/stats

Query params : `?platform_id=uuid`

Response 200 :

```
{
  "platform_id": "uuid",
  "total_active": 1542,
  "total_trial": 89,
  "total_cancelled": 234,
  "mrr": 12500000,
  "plans_breakdown": [
    { "plan": "Free", "count": 980 },
    { "plan": "Pro", "count": 450 },
    { "plan": "Enterprise", "count": 112 }
  ]
}
```

Codes : 200, 401, 403.

9.8.2 POST /subscriptions/admin/{subId}/suspend

Suspension forcée. Codes : 200, 403, 404.

9.8.3 POST /subscriptions/admin/{subId}/force-change

Changement de plan forcé (sans paiement). Codes : 200, 403, 404.

9.8.4 DELETE /subscriptions/by-user/{userId}

Purge RGPD. Auth : S2S token. Supprime abonnements, usage, events du subscriber.

Codes : 200, 401, 404.

10 Contrats Inter-Services

10.1 Abonnement → Auth (JWT utilisateur)

Paramètre	Valeur
Endpoint	`GET /auth/verify-token`
Auth	`Authorization: Bearer <access_token>`
Données extraites	`user_id` (`sub`), `platform_id`
Cache	Redis TTL 30s
Comportement échec	Retourner 401 Unauthorized

10.2 Abonnement → Auth (token S2S)

Paramètre	Valeur
Endpoint	`POST /auth/s2s/introspect`
Données	`client_id` (= `platform_id`), `service_name`
Usage	Appels S2S entrants (quota check depuis Chatbot, Média, etc.)

10.3 Abonnement → Users (résolution subscriber)

Paramètre	Valeur
Endpoint	`GET /api/v1/users/{userId}/status`
Usage	Vérifier que le user est actif avant souscription
Cache	Redis TTL 60s
Comportement échec	Retourner 400 « User not found or inactive »

10.4 Abonnement → Notification (alertes)

Événement	Notification déclenchée
Trial expire dans 3 jours	Email/push : « Votre essai expire bientôt »
Renouvellement réussi	Email : « Votre abonnement a été renouvelé »
Renouvellement échoué	Email/push : « Échec de paiement, action requise »
Entrée en grâce	Email/push : « Abonnement expiré, délai de grâce de X jours »
Suspension	Email : « Votre abonnement a été suspendu »
Quota à 80%	Push/in-app : « 80% de votre quota X consommé »
Quota atteint (hard limit)	Push/in-app : « Quota X atteint, upgradez votre plan »

10.5 Services consommateurs → Abonnement (vérification quotas) (v1.2)

Service	Endpoint appelé	Moment
Chatbot	`POST /quotas/reserve`	Avant chaque réponse IA (chemin critique haute concurrence)
Chatbot	`POST /quotas/confirm`	Après réponse IA réussie
Chatbot	`POST /quotas/release`	Si échec génération IA
Média	`POST /quotas/reserve`	Avant chaque upload (chemin critique haute concurrence)
Média	`POST /quotas/confirm`	Après upload réussi
Média	`POST /quotas/release`	Si échec upload
Search	`POST /quotas/check`	Avant activation boost (faible concurrence, check suffisant)
AGT-Bot	`POST /quotas/check`	Avant création de bot (faible concurrence, check suffisant)

Règle d'usage (v1.2)

Chemins critiques (haute fréquence, concurrence élevée) : Chatbot, Média → **reserve/confirm/release obligatoire**.
Chemins non critiques (faible fréquence, opérations ponctuelles) : Search, AGT-Bot → `check` seul suffit.

10.6 Abonnement → Paiement (événements RabbitMQ) (v1.1)

Événement publié	Payload
`subscription.payment_required`	`event_id` (UUID unique), `idempotency_key` (UUID), `subscription_id`, `amount`, `currency`, `subscriber_type`, `subscriber_id`, `reason` (new, renewal, upgrade), `timestamp`
`subscription.overage_billing`	`event_id`, `idempotency_key`, `subscription_id`, `quota_key`, `overage_amount`, `unit_price`, `total`, `quota_snapshot` (v1.1), `timestamp`

Idempotence (v1.1)

Règle stricte : chaque événement porte un `event_id` (UUID v4) et un `idempotency_key`. Le consommateur (Paiement) **doit** dédupliquer sur `idempotency_key` avant traitement.
Scénarios couverts :
- Retry RabbitMQ après timeout : même `idempotency_key` → ignoré par Paiement
- Double émission (bug ou crash recovery) : détecté et ignoré
- Webhook Paiement doublonné : Abonnement déduplique sur `payment_reference_id`

Stockage : table `processed_events` côté consommateur avec TTL 72h (index sur `idempotency_key`).

Événements reçus de Paiement (v1.1)

Abonnement écoute les queues `payment.confirmed` et `payment.failed` :
- `payment.confirmed` : contient `payment_reference_id`, `subscription_id`, `idempotency_key`. Abonnement vérifie que le `payment_reference_id` n'a pas

déjà été traité (table `subscription_events.metadata`). Si nouveau → activation ou renouvellement.
— `payment.failed` : même structure. Déclenche la logique de grâce ou suspension.

Déduplication : Abonnement stocke chaque `payment_reference_id` traité. Tout doublon est ignoré silencieusement (log warning, pas d'erreur).

10.7 Users → Abonnement (Purge RGPD)

Paramètre	Valeur
Endpoint	`DELETE /api/v1/subscriptions/by-user/{userId}`
Auth	Token S2S
Usage	Appelé par Users dans le workflow de purge RGPD
Données supprimées	Abonnements, usage, events du subscriber. Organisations si owner.
Séquence	Avant Auth purge et Média purge

11 Matrice d'Autorisation

Endpoint	JWT	S2S	Admin Plat.	Admin Global
GET /plans				
POST /plans				
PUT /plans/{id}				
POST /plans/.../archive				
POST /subscriptions				
GET /subscriptions/{id}				
POST /.../cancel				
POST /.../change-plan				
POST /quotas/check				
POST /quotas/increment				
POST /quotas/reserve				
POST /quotas/confirm				
POST /quotas/release				
GET /.../usage				
POST /organizations				
PUT /config/{platformId}				
GET /admin/stats				
POST /admin/.../suspend				
DELETE /by-user/{userId}				

12 Recommandations & Évolutions

— **Coupons & réductions** : module pour codes promo, remises fidélité.
— **Usage analytics** : dashboard de consommation par quota, tendances, prédictions.
— **Webhooks sortants** : événements cycle de vie vers les plateformes.
— **Quotas imbriqués** : quotas par membre d'organisation (en plus des quotas partagés).
— **Multi-abonnement** : add-ons en complément du plan principal.
— **Versioning API** : préfixe `/api/v1`. Breaking changes → `/api/v2`.
— **Monitoring** : métriques Prometheus (souscriptions, quota checks/s, latence P95, taux renouvellement).

Annexes

Glossaire

Terme	Définition
Subscriber	Entité abstraite qui détient un abonnement (user ou organization)
Plan	Offre nommée avec prix, quotas et features, scopée à une plateforme
Quota	Limite mesurable associée à un plan
Overage	Dépassement d'un quota autorisé, facturé à l'unité
Hard limit	Quota dont le dépassement est interdit
Prorata	Calcul au jour du crédit/débit lors d'un changement de plan
Trial	Période d'essai gratuite avant activation payante
Grace period	Délai après expiration où l'accès est maintenu
MRR	Monthly Recurring Revenue
Billing cycle	Période de facturation
S2S	Service-to-Service

Code PlantUML — Diagramme de Classes

Le code PlantUML ci-dessous correspond au diagramme de classes complet du service Abonnement v1.2. Il peut être rendu sur `plantuml.com` ou tout outil compatible.

```
@startuml Subscription Service - Class Diagram v1.2

skinparam classAttributeIconSize 0
skinparam linetype ortho

entity "plans" as plans {
  * id : UUID <<PK>>
  --
  * platform_id : UUID
  * name : VARCHAR(100)
  * slug : VARCHAR(100)
  description : TEXT
  is_active : BOOLEAN = true
  is_default : BOOLEAN = false
  is_free : BOOLEAN = false
  * tier_order : INT = 0
  metadata : JSONB
  * created_at : TIMESTAMPTZ
  * updated_at : TIMESTAMPTZ
  ..
  <<UQ>> (platform_id, slug)
}

entity "plan_prices" as prices {
  * id : UUID <<PK>>
  --
  * plan_id : UUID <<FK>>
  * billing_cycle : VARCHAR(20)
  cycle_days : INT
  * price : DECIMAL(12,2)
  * currency : VARCHAR(3)
  is_active : BOOLEAN = true
  * created_at : TIMESTAMPTZ
  ..
  <<UQ>> (plan_id, billing_cycle, currency)
}

entity "plan_quotas" as quotas {
  * id : UUID <<PK>>
  --
  * plan_id : UUID <<FK>>
  * quota_key : VARCHAR(50)
  * limit_value : INT
```

```
  is_cyclical : BOOLEAN = true
  overage_policy : VARCHAR(10) = 'hard'
  overage_unit_price : DECIMAL(10,2) = 0
  ..
  <<UQ>> (plan_id, quota_key)
}

entity "organizations" as orgs {
  * id : UUID <<PK>>
  --
  * platform_id : UUID
  * name : VARCHAR(150)
  * owner_user_id : UUID
  is_active : BOOLEAN = true
  * created_at : TIMESTAMPTZ
  * updated_at : TIMESTAMPTZ
  ..
  <<UQ>> (platform_id, name)
}

entity "organization_members" as members {
  * id : UUID <<PK>>
  --
  * organization_id : UUID <<FK>>
  * user_id : UUID
  role : VARCHAR(20) = 'member'
  * joined_at : TIMESTAMPTZ
  ..
  <<UQ>> (organization_id, user_id)
}

entity "subscriptions" as subs {
  * id : UUID <<PK>>
  --
  * platform_id : UUID
  * subscriber_type : VARCHAR(20)
  * subscriber_id : UUID
  * plan_id : UUID <<FK>>
  * plan_price_id : UUID <<FK>>
  * status : VARCHAR(20)
  * current_period_start : TIMESTAMPTZ
  * current_period_end : TIMESTAMPTZ
  trial_end : TIMESTAMPTZ
  grace_end : TIMESTAMPTZ
  cancelled_at : TIMESTAMPTZ
  cancel_at_period_end : BOOLEAN = false
  * created_at : TIMESTAMPTZ
  * updated_at : TIMESTAMPTZ
  ..
  <<PARTIAL UQ>> (platform_id, subscriber_type, subscriber_id)
  WHERE status NOT IN ('expired','cancelled')
}

entity "subscription_quotas_usage" as usage {
  * id : UUID <<PK>>
  --
  * subscription_id : UUID <<FK>>
  * quota_key : VARCHAR(50)
  used : INT = 0
  overage : INT = 0
  * period_start : TIMESTAMPTZ
  * period_end : TIMESTAMPTZ
  * updated_at : TIMESTAMPTZ
  ..
  <<UQ>> (subscription_id, quota_key, period_start)
}

entity "quota_reservations" as reservations {
  * id : UUID <<PK>>
  --
  * subscription_id : UUID <<FK>>
  * quota_key : VARCHAR(50)
  * amount : INT
```

```
  status : VARCHAR(20) = 'pending'
  * expires_at : TIMESTAMPTZ
  * created_at : TIMESTAMPTZ
  resolved_at : TIMESTAMPTZ
}

entity "subscription_events" as events {
  * id : UUID <<PK>>
  --
  * subscription_id : UUID <<FK>>
  * event_type : VARCHAR(30)
  from_plan_id : UUID
  to_plan_id : UUID
  prorate_credit : DECIMAL(12,2)
  prorate_debit : DECIMAL(12,2)
  metadata : JSONB
  * created_at : TIMESTAMPTZ
}

entity "platform_subscription_config" as config {
  * id : UUID <<PK>>
  --
  * platform_id : UUID <<UQ>>
  default_trial_days : INT = 0
  grace_period_days : INT = 0
  post_trial_behavior : VARCHAR(20) = 'suspend'
  default_currency : VARCHAR(3) = 'XAF'
  * allowed_cycles : JSONB
  require_default_plan : BOOLEAN = false
  * created_at : TIMESTAMPTZ
  * updated_at : TIMESTAMPTZ
}

plans ||--o{ prices   : "1..* prices"
plans ||--o{ quotas   : "1..* quotas"
plans ||--o{ subs     : "1..* subscriptions"

prices ||--o{ subs    : "1..* subscriptions"

orgs ||--o{ members   : "1..* members"

subs ||--o{ usage     : "1..* usage"
subs ||--o{ events    : "1..* events"
subs ||--o{ reservations : "0..* reservations"

@enduml
```